TECHNICAL WHITEPAPER · V1.3 · PERFORMANCE ENGINEERING

# Moving a 504 out of the system

Read Amplification, Timeout Budget Inversion, and a Stale-While-Revalidate Status Engine. An admin endpoint that answered in 433 ms on a laptop returned `HTTP 504` in production, because a four-fold read amplification only becomes fatal when storage latency crosses from local NVMe to a mounted network volume. This report isolates that defect with a controlled experiment, shows why removing the redundant reads is not sufficient, and moves the scan off the request path entirely. Every figure is reproducible from the archived harness.


🗓️ 2026-10-29 · ✍️ Lee Teong Hoe · Hoelee Enterprise


🧪 Controlled benchmark · three independent full runs

⏱️ Request path 0.3 µs · 📦 Artifact, harness & raw JSON archived

## 01 Abstract


The dangerous property of this defect is not that it is slow. It is that **it is not a defect on the machine where it is written**. On local NVMe the naive implementation is only 1.5–1.9× slower than a single-pass rewrite; the same code, on storage where each read carries a round trip, scales linearly with the read count into the tens of seconds and crosses the outer gateway's response-header budget. That is the standard path to "green locally, 504 in production".


**NAIVE · COLD**

**433.5 ms**

960 whole-file reads over a 360-file corpus

**SINGLE PASS · COLD**

**230.0 ms**

only 1.5–1.9× better, still seconds

**SWR CACHE HIT**

**0.3 µs**

zero filesystem I/O on the request path

**INCREMENTAL REFRESH**

**14.7 ms**

one file changed, 239 rows reused

1. **Read amplification.** One status query performs four separate `read_text()` calls per source file and roughly three YAML frontmatter deserialisations. At 240 source notes that is **960 whole-file reads** for a page that renders titles and three status words.
2. **The defect is coupled to storage latency, not to CPU.** Locally: 433.5 ms against 230.0 ms — 1.61 / 1.50 / 1.89 across three full runs, 1.88× on medians. Projected at 20 ms per read, the naive loop reaches **18.5 s** and the single-pass rewrite still costs **7.9 s**. The rewrite stops the 504 and keeps the eight-second freeze.
3. **The fix is therefore not a faster scan.** It is **moving the scan out of the request path**: requests read an in-memory snapshot, staleness is bounded by a 60-second TTL, and a background single-flight refresh maintains it.

> The acceptance criterion after the rewrite is not "how much faster". It is **whether any filesystem I/O remains on the request path**. A second criterion sits next to it, because caching most often removes a business capability rather than latency: the `ready / outdated / pending` classification is a pass/fail item, not a cosmetic one.

---

## 02 Incident and read amplification

### 2.1 The path the request takes

An ordinary multi-layer reverse proxy chain. Read bottom-up it looks like generous headroom at every hop; read top-down it is a trap, and section 03 is the reason. The timeout budget of each layer is printed on the right.

**Browser / admin console** `HTTPS :443`

▼ TLS termination and routing

**Edge gateway · Traefik v3** `responseHeaderTimeout = 30s`

▼ internal docker network, identity headers injected before forwarding

**Identity gateway · forward-auth** `forward auth provider`

▼ reverse proxy header injection

**Backend reverse proxy · Nginx (static / API split)** `proxy_read_timeout = 60s`

▼ HTTP / unix socket

**Application · Flask backend** `gunicorn --timeout 120s`

▼ synchronous filesystem walk: `videos/*.md` plus `videos/*.en.md`, note by note

### 2.2 What the console shows

The console fetches the status through a normal async call; the gateway's 504 surfaces as a persistent error banner. Nothing in the front end is broken — it is faithfully reporting a back-end defect.

```
Failed to read translation status: Error: HTTP 504 Gateway Time-out
```

### 2.3 One query, four reads

The route calls `list_status()`. For every source file it asks four questions and answers each one by opening the file again: which locale, what is the hash, what is the title, and is a translation current.

```
def list_status(content_dir, locale="zh"):
    for p in Path(content_dir).glob("*.md"):
        if p.name.endswith(".en.md"):
            continue
        raw = p.read_text(encoding="utf-8")       # read 1: which locale?
        meta, body = split_note(raw)
        if meta.get("locale") != locale:
            continue
        results.append(status(p))

def status(source_path):
    src_hash = hash_of_note(source_path)          # read 2: hash it
    meta, _ = split_note(source_path.read_text(encoding="utf-8"))   # read 3: title
    target = source_path.with_name(f"{source_path.stem}.en.md")
    if target.exists():
        tgt_hash = hash_of_note(target)           # read 4: compare translation
```

**Amplification.** Four reads per source file plus a fifth for every translated note. At 240 notes: **960 whole-file reads** and roughly 720–960 YAML deserialisations per request. The loop is not wrong per iteration; it asks the same question four times because each helper was written to be self-sufficient. Section 05 shows that on local disk this costs 1.5–1.9×, and that on a mounted volume it scales with the read count.

## 03 Timeout budget inversion

The request did not fail because it was slow. It failed because the outer layer gave up first and nothing downstream noticed. When one layer is configured to be more patient than the layer in front of it, the extra patience does not buy headroom — it buys work that nobody collects.

| Layer | Directive | Value | Behaviour |
|---|---|---|---|
| **Edge gateway** | `forwardingTimeouts.responseHeaderTimeout` | `30 s` | Closes the client connection and returns 504 if no upstream response header arrives in 30 s |
| **Backend reverse proxy** | `proxy_read_timeout` | `60 s` | Keeps waiting on the application, up to one minute |
| **Application container** | `--timeout` | `120 s` | Still holding a worker, CPU and storage bandwidth with no awareness of the abandonment |

**Physical evidence.** The same `X-Request-ID` appears in the gateway log marked *abandoned* and in the application log marked *completed 200*, seconds apart. Because the client only ever saw the 504, the request's true duration is not observable from either side alone — and **a duration that cannot be measured from both logs is a duration that should not be quoted**.

Inversion is usually introduced by kindness: during an incident somebody raises the timeout so the "slow" endpoint stops erroring, and raises it at whichever layer surfaced the error. The result is a stack whose innermost layer is the most patient, and therefore the most wasteful.

### 3.1 Why a single-pass rewrite is not a fix

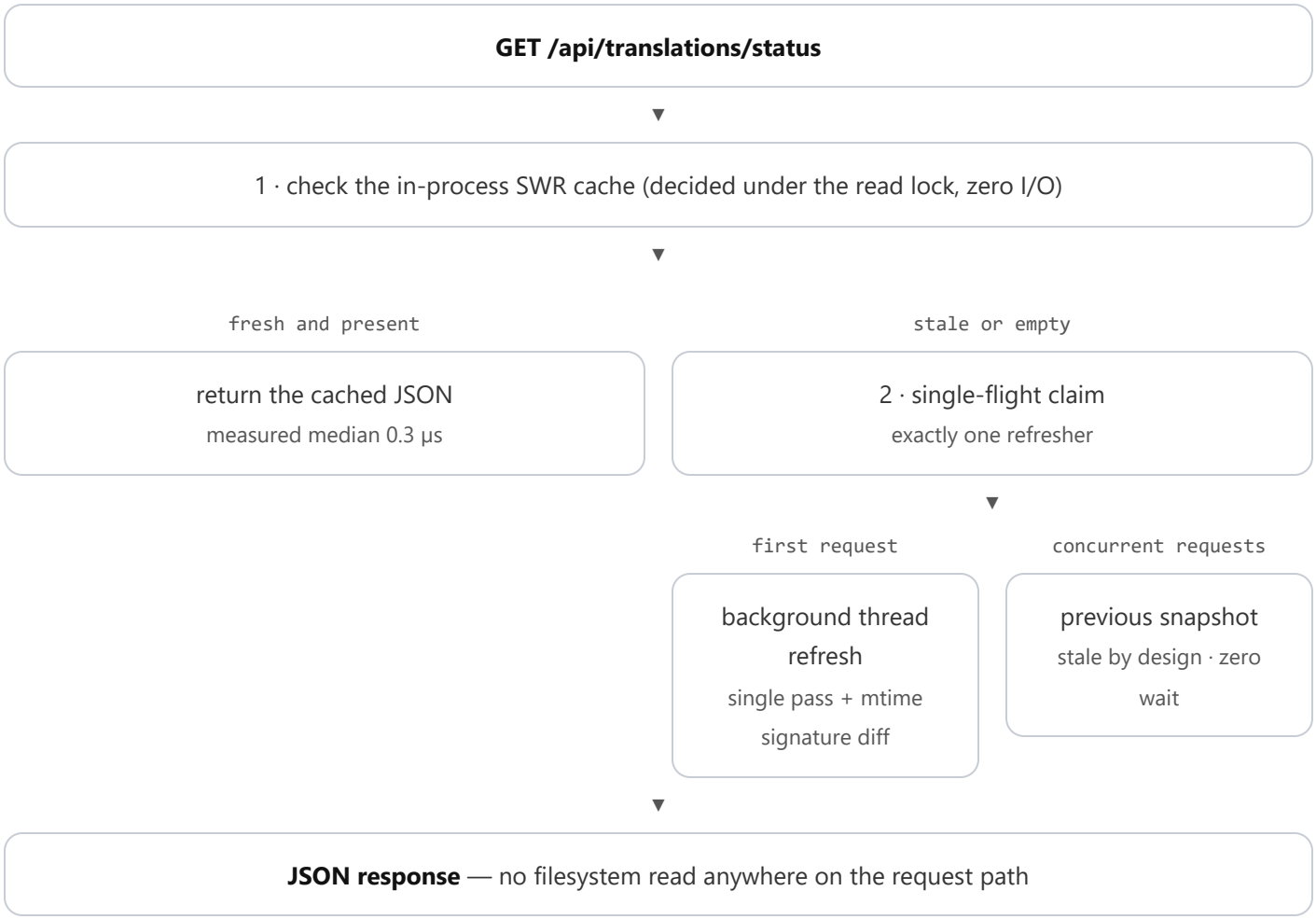
Collapsing four reads into one is measurable and worth doing, but it is a constant-factor change. At 20 ms per read the cold scan is still 7.9 s. That no longer returns a 504 and still hands the operator an eight-second freeze, repeated at every TTL expiry.

> **The judgement generalises to every endpoint that walks a filesystem:** as long as the scan is on the request path, optimisation moves the constant, not the structure. The structure is that the response cannot be produced until the scan finishes.

## 04 Refactor: SWR engine with single-flight refresh

One sentence describes the target: **requests always read memory, scans always happen in the background**. Fresh means return directly; stale means claim the refresh, start a background thread, and return the previous

snapshot to this caller; concurrent requests do not queue behind the scan, they take the old value.




### 4.1 Core implementation

Three things are written inside one critical section: the snapshot, the signature map, and the refresh latch. The signature map is the entire source of incrementality — an unchanged `(st_mtime_ns, st_size)` pair means **zero `read_text()` calls** and the previous row is reused.

🔍 **View the SWR cache, single-flight latch and mtime diff engine (Python, 68 lines)**

```
import threading, time
from pathlib import Path

_TTL = 60.0
_CACHE = {"data": None, "stamp": 0.0, "refreshing": False, "mtime_map": {}}
_LOCK = threading.Lock()       # guards _CACHE itself
_COLD_LOCK = threading.Lock()  # cold start: one scanner, not a stampede

def get_status(content_dir, locale="zh"):
    with _LOCK:
```

```
        has_data = _CACHE["data"] is not None
        if has_data and (time.time() - _CACHE["stamp"]) < _TTL:
            return _CACHE["data"]                    # fresh: zero I/O
        if has_data and _CACHE["refreshing"]:
            return _CACHE["data"]                    # already refreshing
        if has_data:
            _CACHE["refreshing"] = True
            threading.Thread(target=_refresh_bg, args=(content_dir, locale),
                             daemon=True).start()
            return _CACHE["data"]                    # claim, then serve stale
    with _COLD_LOCK:
        with _LOCK:
            if _CACHE["data"] is not None:
                return _CACHE["data"]
        return _refresh_now(content_dir, locale)

def _refresh_now(content_dir, locale):
    rows, new_map = _scan(content_dir, locale)
    with _LOCK:
        _CACHE["data"] = rows
        _CACHE["mtime_map"] = new_map  # data and signatures: one lock, one write
        _CACHE["stamp"] = time.time()
        _CACHE["refreshing"] = False
    return rows

def _refresh_bg(content_dir, locale):
    try:
        _refresh_now(content_dir, locale)
    except Exception:
        with _LOCK:
            _CACHE["refreshing"] = False  # never leave the flag stuck on failure

def _scan(content_dir, locale):
    rows, new_map = [], {}
    for p in sorted(Path(content_dir).glob("*.md")):
        if p.name.endswith(".en.md"):
            continue
        st = p.stat()
        tgt = p.with_name(f"{p.stem}.en.md")
        ts = tgt.stat() if tgt.exists() else None
        sig = (st.st_mtime_ns, st.st_size,
               ts.st_mtime_ns if ts else None, ts.st_size if ts else None)
        seen = _CACHE["mtime_map"].get(p.name)
        if seen and seen[0] == sig:      # unchanged: no read_text at all
            rows.append(seen[1]); new_map[p.name] = seen
            continue
        raw = p.read_text(encoding="utf-8")
        meta, body = split_note(raw)
        if meta.get("locale") != locale:
```

```
            continue
        row = {"slug": p.stem, "title": meta.get("title", p.stem),
               "source_hash": hash_memory(meta, body), "status": "pending"}
        if ts:
            tgt_meta, _ = split_note(tgt.read_text(encoding="utf-8"))
            row["status"] = ("ready" if (tgt_meta or {}).get("translatedFromHash")
                             == row["source_hash"] else "outdated")
        rows.append(row); new_map[p.name] = (sig, row)
    return rows, new_map
```

### 4.2 Three real defects fixed against the first version

1. **The signature map became an actual mechanism.** The first version declared `mtime_map` and never read it: every background refresh still called `read_text()` on every file. Keying the cache on the two-part signature makes an unchanged file free. Measured: with one note changed out of 240, the refresh reads exactly one file.
2. **Three-state classification was restored.** The first version reduced "is this translation current?" to "does the translation file exist?", which deletes the business capability. The comparison that works is against the source hash *recorded at translation time* (`translatedFromHash` in the translation's own front matter).

   > This one started as my own mistake: comparing the **source body hash** with the **translated body hash** can never be equal — they are different documents in different languages. The benchmark surfaced it, because the first corpus run reported `0 ready` out of 60 expected, a number no rational generator produces. A harness that only reported milliseconds would have shipped a 230 ms regression.

3. **Cold-start stampede was closed.** The first version scanned the whole directory whenever the cache was empty, so five concurrent requests arriving at a cold cache each launched their own walk. A separate cold-start lock makes the first request scan and the rest queue for its result. v1.2 additionally writes the snapshot and the signature map inside the same critical section, so a reader can never observe new rows paired with an old signature table.

### 4.3 Multi-worker evolution: four paths, ordered by operational cost

The state above is **process-local**. Under 2–4 workers each worker scans once and keeps its own TTL. The four options, cheapest operationally first:

| Option | Cross-process latency | External dependency | When it wins |
|---|---|---|---|
| **Per-worker cache + jittered TTL** | `0 (local memory)` | none | ≤ 4 workers and ≤ 500 notes: worst case four duplicate scans, request path still microseconds |
| **tmpfs snapshot + `fcntl.flock`** | `< 0.5 ms` | none | Single host, several workers, and a hard requirement that all workers agree; the container needs an explicit mount |
| **SQLite (WAL) state table** | `~1 ms` | none | State must survive restarts, or must be auditable |
| **Dedicated Redis** | `~0.3 ms` | yes | Only when Redis already exists; adding middleware to hold one status table does not pay for itself |

For a container using the tmpfs option: Docker allocates **64 MB** by default, and a 1,000-note status snapshot is roughly 200 KB, so the default is not a reason to reject the design.

This project chose the first option — **no new component, duplicate scans accepted**. The operation is triggered by hand, its consistency requirement is eventual, and a 60-second TTL absorbs the difference. Adding Redis would convert a local filesystem problem into a new availability dependency: a larger component solving a smaller problem.

---

# `05` Controlled benchmark

## 5.1 Method

- **Corpus.** Programmatically generated: 240 source notes plus 120 translations, 360 `.md` files, 1,315,740 bytes, 12 body paragraphs and YAML front matter each. 60 translations record the current source hash (`ready`), 60 sources were edited after translation (`outdated`), 120 have no translation (`pending`).
- **Environment.** Local NVMe SSD, Windows 11, Python 3.14.9, single process. **This is not the production environment**: the experiment quantifies the relative cost of read amplification, it does not reproduce NAS latency.
- **Procedure.** Each implementation is warmed for one round, then measured over five consecutive rounds and reported by median. Cache-hit latency is taken from 200 timed calls, reported as median and P99. The **entire experiment is run independently three times**; the table reports the median of those three runs and the min–max range, so machine jitter is visible rather than averaged away.
- **Reproducibility.** The harness is archived with this record together with the three raw result sets of a 2026-10-30 replication session and the two pre-correction v1.1 outputs discussed in 5.4. Session A's own runs (the table in 5.2) were transcribed from console output rather than stored — which is precisely why 5.5 repeats the experiment instead of asserting it. Harness stdout is strictly valid JSON with no local paths, so any run can be diffed in CI.

### 5.2 Measured readings (local NVMe, three full runs)

| Metric | Median | Range across three full runs |
|---|---|---|
| Naive (4 reads + 3 YAML parses) · cold | **433.5 ms** | 333.8 – 452.1 ms |
| Single pass (1 read + 1 parse) · cold | **230.0 ms** | 222.6 – 280.9 ms |
| SWR cold start (stat + signature map build) | **263.1 ms** | 202.5 – 275.1 ms |
| SWR cache hit (request path) | **0.3 µs** | P99 2.8 – 7.8 µs |
| SWR incremental refresh (TTL expired, one note changed) | **14.7 ms** | 14.53 – 15.74 ms |
| Three-state classification correctness | **ready 60 / outdated 60 / pending 120** | identical in all three runs |

**The key reading is how small the improvement is.** Removing three quarters of the reads bought **1.5–1.9×** (1.61 / 1.50 / 1.89 per run; 1.88× on medians) — nowhere near the intuitive "four reads, four times the cost". On local storage the disk was never the bottleneck: the syscall overhead disappeared into the page cache and what remained was parsing and CPU. A four-fold read multiplier that costs 433 ms passes review, passes tests, and ships. It is only wrong on storage where every read carries a round trip.

Equally, the machine itself jitters by **±20%** between full runs of the same implementation. The report therefore quotes medians and ranges and never a single value to one decimal place — that precision does not exist in this environment. Section 5.5 measures the same variance *between* sessions and finds it larger: 19–27%.

### 5.3 What incrementality is worth

Incremental refresh (14.7 ms) is **15.6×** faster than a cold single-pass scan (230.0 ms; 14.6–19.3× per run) and about **29.5×** faster than the naive loop (433.5 ms). Both multipliers are session-A values; the 2026-10-30 replication measured 11.5× and 20.3× (5.5), which is why neither is stated here as a specification. The multiplier is not the point. The structural fact is that incremental refresh is **not on the request path**: it runs in a background thread and the caller receives the previous snapshot.

### 5.4 Network-volume projection (model output, not a measurement)

The projection is not produced by multiplying the baseline by a read count. `Path.read_text` **is intercepted and made to sleep for L, then each implementation is re-run end to end.** The resulting figures therefore include Python call overhead and the actual granularity of `sleep`, and deliberately do not reconcile exactly with "baseline + reads × L" — a reader who recomputes that way will see a systematic offset of a few hundred milliseconds. That offset is the method, not an error in the table. The rows correspond to 960 / 360 / 360 `read_text` calls (plus roughly 480 `stat` calls) and to 2 reads for the incremental path.

| Cost per read L | Naive (960 reads) | Single pass (360 reads) | SWR cold start | SWR incremental (1 file changed / 2 reads) |
|---|---|---|---|---|
| **0 ms (measured)** | 433.5 ms | 230.0 ms | 263.1 ms | 14.7 ms |
| **5 ms** | 5,552 ms | 2,347 ms | 2,433 ms | 18.5 ms |
| **10 ms** | 9,654 ms | 4,244 ms | 4,239 ms | 30.2 ms |
| **20 ms** | **18,508 ms** | 7,898 ms | 7,857 ms | 36.9 ms |

> **One experimental defect corrected in v1.2.** The v1.1 table filled its last column with the measured "one file changed" figure in the 0 ms row, but with the "no change / 0 reads" projection in the 5/10/20 ms rows — so the column got *faster* as the read penalty grew, which is physically impossible. The projection module now touches one note before measuring, making all four rows the same test condition: the incremental column rises monotonically (14.7 → 18.5 → 30.2 → 36.9 ms) and always performs exactly two `read_text` calls.

**How to read it.** The naive implementation's cost is nearly proportional to its read count. At 30 ms per read — unremarkable for a busy SMB mount — the I/O alone is 960 × 30 ms ≈ 28.8 s, past a 30-second response-header budget before parsing begins. That matches the production symptom. The incremental path reads one file and stays in the tens of milliseconds at every L.

> **Limits of this projection, stated plainly.** The model bills `read_text()` and does not bill `stat()`. On SMB/NFS every `stat` is also a round trip, and the incremental path's dominant cost is roughly 480 of them. At 5 ms per `stat`, incremental refresh is closer to **2.4 s** than to 37 ms. It is still off the request path, so the 504 conclusion holds — but "tens of milliseconds" would be an overstatement, and this report does not make it.

### 5.5 Cross-session replication (2026-10-30)

Every number in 5.2 and 5.4 comes from session A. One session proves nothing about reproducibility, so the same harness was run three more times on the same hardware the following day: same corpus generator, no GPU workload, no other benchmarks running. These three runs are the raw JSON archived with this record.

| Replication run | Naive · cold | Single pass · cold | SWR cold start | Cache hit (median / P99) | Incremental | Naive ÷ single |
|---|---|---|---|---|---|---|
| **Run 1** | 461.7 ms | 242.7 ms | 293.7 ms | 0.3 / 3.1 µs | 29.51 ms | 1.90× |
| **Run 2** | 517.1 ms | 292.2 ms | 333.1 ms | 0.6 / 6.4 µs | 25.42 ms | 1.77× |
| **Run 3** | 559.5 ms | 306.6 ms | 357.2 ms | 0.5 / 3.5 µs | 25.50 ms | 1.82× |
| **Median** | **517.1 ms** | **292.2 ms** | **333.1 ms** | **0.5 / 3.5 µs** | **25.50 ms** | **1.77×** |

Four findings, one of them uncomfortable:

- **The structure replicates exactly.** Cache hits stay sub-microsecond (median 0.5 µs, P99 3.5 µs) and incremental refresh reads exactly one file in every run. **Zero filesystem reads remain on the request path** — the property the refactor exists to create, and the only claim here that does not depend on timing.
- **The cold-path order replicates.** The naive loop costs 1.77–1.90× the single-pass rewrite in every replication run (session A: 1.50–1.89×). Removing three quarters of the reads buys less than 2×, locally, in both sessions.

- **The absolute values do not replicate.** This session ran 19% / 27% / 27% slower than session A on the naive, single-pass and SWR-cold paths. Cross-session drift is therefore *larger* than the ±20% within-session band — a median is a property of a session, not a constant. That is the report's own argument, applied to the report's own table.
- **The incremental metric is the least stable, so 5.3's multiplier is the softest number in this report.** 14.7 ms in session A against a 25.5 ms median in replication (25.4–29.5). Its cost is dominated by roughly 480 metadata calls whose price depends on cache state — precisely the term 5.4 says the projection does not bill. The multiplier is 15.6× in session A and 11.5× here; it exceeds 10× in both, and the conclusion (the scan is off the request path) holds in both.

The replication also drifts *within* the session: 461.7 → 517.1 → 559.5 ms across three consecutive runs, monotonically slower. On a thermally throttling laptop a benchmark is a moving target, and reordering runs does not fix it. It is reported rather than smoothed away.

---

## 06 Conclusions and known limits

1. **A performance defect coupled to storage latency produces false negatives in local benchmarks.** 1.5–1.9× on NVMe against tens of seconds on a mounted volume is the same code. Capacity planning must use the slowest storage the code will ever run on, not the machine it was written on.
2. **Timeout budgets must decrease toward the innermost layer.** 30 s / 60 s / 120 s provides no headroom; it manufactures orphaned responses — compute already spent, result already discarded, and neither log able to state the duration alone.
3. **Moving a slow operation out of the request path beats making it faster.** 433.5 ms became 0.3 µs not through a better algorithm, but by deciding that this second the request will not touch a disk at all.
4. **Cold start remains the most expensive scan and must be pre-warmed.** The container issues one background scan at startup, so the single 263 ms cost (several seconds on a network volume) is paid before the process reports ready rather than at the first click.
5. **The main risk of caching is that a business capability disappears silently.** The first version's regression — a three-state display collapsing into two — is invisible to any latency test and surfaces days later as a user complaint. The classification result therefore sits in the results table next to the latency numbers.
6. **Consistency is server-side write-through only; no client-controlled refresh switch.** Saving a translation calls `invalidate(slug)`. There is deliberately no `?force_refresh=1`: every client-controlled bypass of the cache is an attack surface that can be pointed at the disk.

> **Known boundary, not fixed by design.** Write-through invalidation assumes the save path always calls `invalidate(slug)`. A translator dropping files in via rsync or an editor bypasses it, and the cache can serve a stale state for up to 60 seconds. This project accepts that eventual-consistency window and does

> not add a `stat` probe on the `GET` path — adding one would invite back the filesystem I/O the whole refactor removed.

---

## 07 References and reproduction

Eight sources, each cited for a specific mechanism used in this report — not as a literature survey. The caching semantics come from the HTTP specifications, the refresh-coordination pattern from the Go ecosystem where it is standard, and the per-operation round-trip cost from the filesystem protocols themselves.


[1] Nottingham, M. *RFC 5861: HTTP Cache-Control Extensions for Stale Content*. IETF, May 2010.
`https://www.rfc-editor.org/rfc/rfc5861.html`

Defines `stale-while-revalidate`, the semantics this engine implements (serve stale, refresh in the background).

[2] Fielding, R., Nottingham, M., Reschke, J. *RFC 9111: HTTP Caching*. IETF, June 2022.
`https://www.rfc-editor.org/rfc/rfc9111.html`

Freshness lifetime and age calculation — the vocabulary behind the 60-second TTL.

[3] Go Project. *Package singleflight (golang.org/x/sync/singleflight)*.
`https://pkg.go.dev/golang.org/x/sync/singleflight`

The canonical statement of the duplicate-suppression primitive: concurrent callers share one in-flight result.

[4] Fitzpatrick, B., et al. *groupcache*.
`https://github.com/golang/groupcache`

Origin of the single-flight cache-fill pattern and the cache-stampede problem it exists to prevent.

[5] Callaghan, B., Pawlowski, B., Staubach, P. *RFC 1813: NFS Version 3 Protocol Specification*. IETF, June 1995.
`https://www.rfc-editor.org/rfc/rfc1813.html`

Each file operation is a separate RPC; the basis for billing per-operation round trips rather than per byte.

[6] Noveck, D., ed. *RFC 8881: Network File System (NFS) Version 4.1 Protocol*. IETF, August 2020.
`https://www.rfc-editor.org/rfc/rfc8881.html`

Confirms that metadata operations carry the same round-trip structure as reads, which is why the projection under-bills `stat`.

[7] Microsoft. *MS-SMB2: Server Message Block (SMB) Protocol Versions 2 and 3*. Open Specifications. `https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/`

The mounted-volume case in this report: request/response framing over the network, one round trip per operation by construction.

[8] Python Software Foundation. *pathlib — Path.stat() and os.stat()*. Python 3 documentation. `https://docs.python.org/3/library/pathlib.html#pathlib.Path.stat`

Establishes `stat` as a distinct metadata call, not folded into `read_text()` — the gap between the model and a real SMB mount.

### 7.1 Reproducing the numbers

```
python swr_bench.py     # corpus generation + three implementations + projection module
                        # stdout: strictly valid JSON, no local paths - diffable in CI
```

The harness generates its own corpus, so it runs anywhere within roughly ±30% hardware variance. Every figure in this report came out of the run it describes — including the four figures that made an earlier version of this work look better than it was, before they were removed and the experiment was re-run.

### 7.2 Citation

```
Lee Teong Hoe. Moving a 504 Out of the System: Read Amplification, Timeout Budget
Inversion, and a Stale-While-Revalidate Status Engine. Technical report v1.3.
Hoelee Enterprise, 2026-10-30. https://doc.hoelee.com/whitepaper-504-read-amplification-swr-en.html
```

---

**Hoelee Enterprise** · software engineering for Malaysian SMEs · blog.hoelee.com

Architectural patterns described here were engineered by the author; client identifiers and production thresholds are sanitised for operational security. The latency projection is a model output, not a production measurement, and the conclusions are bounded by the limitations stated in sections 5.4 and 6.

Technical report v1.3 · 2026-10-30 · first version v1.0 2026-10-28 · data treatment converged through three independent review rounds